Project Milestone 4

Project title: Vgc Matchup Predictor

Team / Author(s): Analytic boost: Abbas Syed, Ethan Jallim, Shan Jeofry

1. Current Methods and Approaches

1.1 Data

Item	Details
Source(s)	[Dataset name, collection method, licence] Name: cameronangliss/vgc-battle-logs Collection: Scrapped through pokemon showdown battle logs (allowed by pokemon showdown through their free use policy)
Size	Processed m-a 6509 entries 24 features Processed m-b 324 entries 24 features
Split strategy	80/ 20 train/test split on Regulation M-A. The validation est is Regulation M-B
Class balance	53 p1 wins, 47 p2 wins
Known quality issues	Nature values don't exist for a majority of dataset M-A

Preprocessing and feature engineering:
The dataset contained logs of battles on showdown. To get information into a useful state, a script was made to parse the logs within the dataset to return the pokemon of each team, and their stats, abilities, moves, and natures. We applied MultiLabelBinarizer via sci kit learn as a few of our labels are arrays of categorical data. Logistic regression was fit on the training group while validation and test were not.

1.2 Models

Stage	Model	Purpose	Key hyperparameters
Baseline	Logistic regression majority class.	Lower bound for comparison	L2 regulizer, C
Intermediate	Random Forest	Strong classical benchmark	max_Depth, n-iter
Main	Fined tuned Logistic Regression	Primary candidate	L2 regulizer, C

1.3 Training and Evaluation Setup

- **Framework and tooling:** scikit-learn. Git as version control software
- **Hardware:** CPU: AMD Ryzen 7 7730 GPU: AMD RADEON GRAPHICS,
- **Loss function and optimiser:** Log loss, L2 Regulizer
- **Regularisation:** L2 ridge regularization
- **Hyperparameter search:** grid search
- **Evaluation metrics:** Primary: ROC-AUC since our model is a binary classifier
- **Reproducibility:** Random seed: 42, requirements.txt has all the required modules for the environment so pip install -r requirements.txt will recreate the environment

2. Theoretical Justification for Chosen Methods

2.1 Choice of Model Family

Logistic regression: Logistic regression is well fitted for binary classifiers and datasets with high dimensionality. Our Data target label is binary classification either p1 wins (positive class) or p2 wins (negative class).

Random forest: Random forest allowed for the logistical thinking of patterns while preventing overfitting that decision trees are prone to do overfitting

2.2 Choice of Loss Function and Optimisation

Log loss is the loss function of choice. For logistic regression models log loss heavily punishes wrong answers which is ideal because there are only 2 possible answers. If an answer is confidently wrong, it punishes it heavily and unsure answers are also punished.

Scikit's default optimizer for logistic regression (Limited-memory Broyden–Fletcher–Goldfarb–Shanno or lbfgs) was used for optimization which uses a L2 regularizer and balances it. It saves on memory allowing for the model to run on less powerful computers. The large amount of features for the data means large complexity which results in large weights. The L2 regularizer punishes these weights

2.3 Bias-Variance and Generalisation

The dataset classes were relatively balanced to begin with as it was a 53 47 class split. Scikits logistic regression uses an L2 regressor and automatically balances it.

2.4 Evaluation Methodology

Since the models are binary classifiers ROC-AUC will give a good estimate on how well it can determine a class. Our dataset is relatively balanced so accuracy and precision are good secondary metrics.

2.5 Alternatives Considered

Alternative	Why it was not chosen
Decision tree	Prone to overfitting

Alternative	Why it was not chosen
Neural network	Time constraints

3. Preliminary Results and Findings

3.1 Quantitative Results

Model	Train [AUC]	Validation [AUC]	Test [AUC]	Training time
[Baseline] Logistic	0.5501	0.5035	0.5844	0.017007sec
[Intermediate] Random Forest	0.6011	0.4843	0.5995	0.195243 sec

3.2 Visualisations to Include

[Insert figures here with captions, e.g. *Figure 1: Validation loss across epochs for the main model.*]

Logistic regression

Figure1 Confusion Matrix

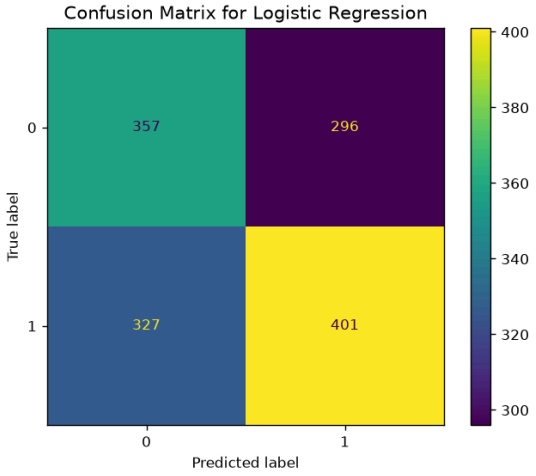
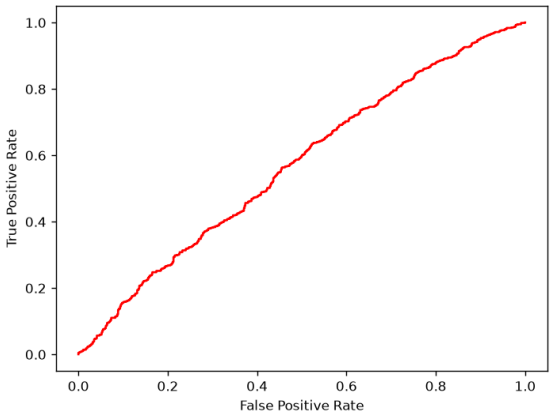
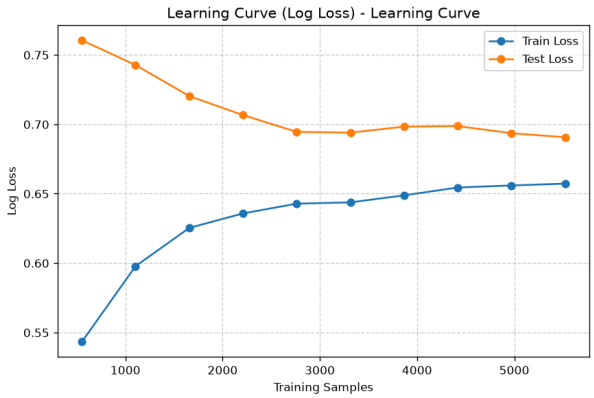
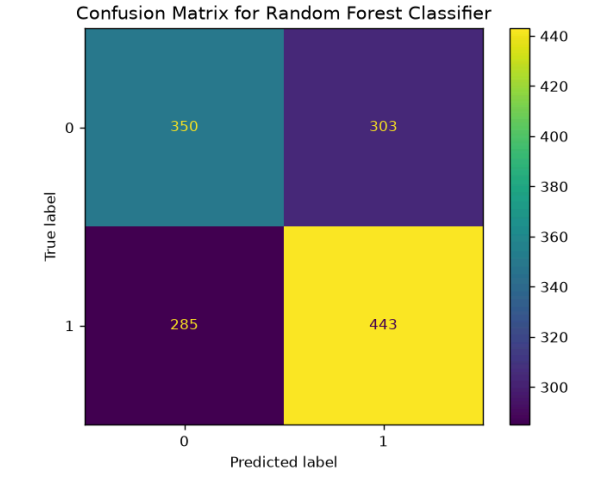


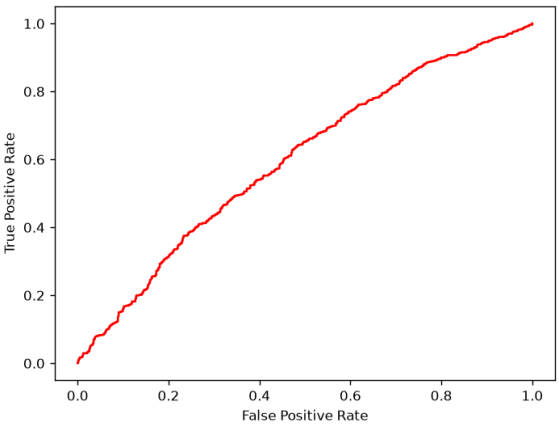
Figure2 ROC Curve

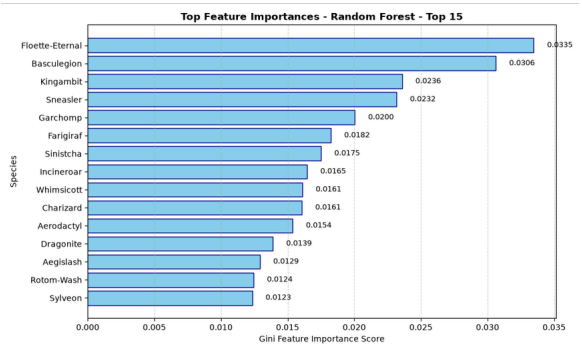


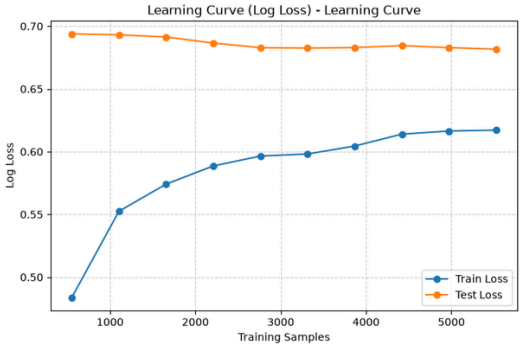


Random Tree Classifier









3.3 Key Findings

1. Using a OneHotEncoder instead of MultilabelBinarizer increased accuracy and precision however decreased AUC score.

3.4 Error Analysis

1. Models evaluation statistics dip during validation, this could be because validation is on reg m-b which contains pokemon that aren't available in regulation m-a and hurt the performance of the model

5.5 Limitations of Current Results

Untuned Baselines: Right now the model builds on pokemon specifically, nothing on their types or speed stat, which are significant stats when determining match up. They were not implemented due to dataset not containing such information

4. Deviations from the Original Plan

Planned	What actually happened	Reason	Impact
Decision Tree model	Random Forest Classifier	Reduce overfitting and	

If there were no deviations, state this explicitly and note any minor adjustments.

5. Current Blockers and Challenges

Challenge	Severity	Effect on project	Mitigation / support needed	Owner
No types and speed to data	Med	Reduces models effectiveness	Second data set to match the given dataset	Shan
Main model	High	Better classifier results		Abbas

6. Revised Timeline and Next Steps

6.1 Revised Timeline

Phase	Tasks	Original date	Revised date	Status
Data collection and cleaning	[Tasks] - Downloading dataset Parsing dataset for useful information	[Date] Oct 2 2026		Complete
Baseline and initial models	[Tasks] Majority class classifier	[Date] Oct 19 2026		Complete
Intermediate model	[Tasks] Random Tree Classifier	[Date] Oct 30 2026		Completed
Model improvement and tuning	[Tasks] Add types and speed stats	[Date] Nov 18	[Date] Nov 21	In progress
Final evaluation on held-out test set	[Tasks] Tune hyperparameters on logistic model	[Date] Nov 24	[Date] Nov 29	Not started
Final report and presentation	[Tasks]	[Date]	[Date]	Not started

Phase	Tasks	Original date	Revised date	Status
	Written Report Video reflection	Dec 4	Dec 6	

6.2 Prioritised Next Steps

1. **[Highest priority]:** Shan: Add types and speed stat to give model better information to work off of and increase performance, target Date: Nov 7.
2. **[Second priority]:** Abbas: Implement main model, Target Date: Nov 15.

6.3 Risks to the Revised Plan and Contingencies

- **If Main model is incomplete :** submit current work and estimate/ report on deeper model

7. Midpoint Peer Check-In

Peer / group consulted: group 11 **Date and format:** Oct 31, written message

7.1 What We Shared

The entirety of the git repository pre-m4. Logistic model and majority classifier benchmark, documentation problem framing.

7.2 Feedback Received from Peers

- **Strengths:** The main advantages are that the repository is well organised, featuring a src/ folder structure, a tests/ directory, pinned dependencies and an editable install procedure. It provides both a logistic regression baseline and a majority-class benchmark so that any subsequent model has a benchmark to compare against.
- **Gaps:** The README includes no metrics and gives no information about the source, size or characteristics of the dataset, making it impossible for a reader to assess the quality of the model. Neither the method of dividing the data into training and test sets nor the validation approach is explained. The setup instructions deal only with activating the program on Windows. There are no mentioned random seeds or expected outputs. Additionally, the README contains no images or diagrams, so a reader cannot at a glance see the data or the results.
- **Suggestions:** Include a results table that compares the benchmark and the baseline on the same data split. You should document the data source and the feature set, and make it clear that no in-battle information is used. Use a more powerful non-linear comparison model, for example a tree ensemble. Also provide the Linux and macOS setup commands.

7.3 Feedback We Gave to Peers

- **Peer project:** Graph Structure as Predictive Signal in Bitcoin Money-Laundering Detection

- **Observations:** The biggest strength I saw in this project would be their consideration of class imbalance. The team appropriately recognized that accuracy by itself is not a reliable evaluation metric for their dataset, and opted to emphasize precision, recall and F1 score. They chose to use a time based train/test split which helps better represent how a real world fraud detection system would operate.

  The dataset selected and their data analysis is very strong in this project. The visualizations provide very useful information about class distribution, transaction connectivity, feature correlations, and the differences between licit and illicit transactions.

  ways they could improve their project, there are a few. Additional details for the graph construction, reverse edges, and how the information leakage will be prevented when training would help a lot. They could also clarify the number of input features used throughout the different milestones.

  Another thing we noticed was some inconsistencies between the milestone documentation and implementation. For example, the baseline notebook initially describes Logistic Regression, while the actual implementation used a Decision Tree. The repo's requirements file could also include all dependencies needed to run the notebooks independently. Minor spelling errors

7.4 Changes Resulting from the Check-In

Add more to README in terms of model metrics and actions

8. Conclusion

Significant progress has been made as both baseline and intermediate models have been implemented. The top risk is finishing the main model with better than current models performance before the due date.

References

1. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html